



SecureApp Pipeline - DAST Report

OWASP ZAP spider + active scan results

Site: <http://secureapp:5000>

Generated on Sat, 8 Aug 2026 20:36:18

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	4
Medium	3
Low	3
Informational	2
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	3
Info	Informational		Percentage of network failures	1 %
Info	Informational	http://secureapp:5000	Percentage of responses with status code 2xx	55 %
Info	Informational	http://secureapp:5000	Percentage of responses with status code 3xx	20 %
Info	Informational	http://secureapp:5000	Percentage of responses with status code 4xx	16 %
Info	Informational	http://secureapp:5000	Percentage of responses with status code 5xx	6 %
Info	Informational	http://secureapp:5000	Percentage of endpoints with content type text/html	100 %
Info	Informational	http://secureapp:5000	Percentage of endpoints with method GET	69 %
Info	Informational	http://secureapp:5000	Percentage of endpoints with method POST	30 %
Info	Informational	http://secureapp:5000	Count of total endpoints	13
Info	Informational	http://secureapp:5000	Percentage of slow responses	18 %

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Cross Site Scripting_(Reflected)	High	1
Path Traversal	High	1
SQL Injection	High	4
SQL Injection - Authentication Bypass	High	1
Absence of Anti-CSRF Tokens	Medium	5
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Missing Anti-clickjacking Header	Medium	Systemic
Cookie without SameSite Attribute	Low	1
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	Systemic
X-Content-Type-Options Header Missing	Low	Systemic
Authentication Request Identified	Informational	1
Session Management Response Identified	Informational	2

Alert Detail

High

Cross Site Scripting (Reflected)

Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology.

When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise.

Description

There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based.

Non-persistent attacks and DOM-based attacks require a user to either visit a specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash.

Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of an attacker's favorite targets often include message board posts, web mail messages, and web chat software. The unsuspecting user is not required to interact with any additional site/link (e.g. an attacker site or a malicious link sent via email), just simply view the web page containing the code.

URL

<http://secureapp:5000/post/new>

Node Name

http://secureapp:5000/post/new {}(content,title)

Method

POST

Parameter	content
Attack	<code></p><script>alert(1);</script><p></code>
Evidence	<code></p><script>alert(1);</script><p></code>
Other Info	
Instances	1
Solution	Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket. Phases: Implementation; Architecture and Design Understand the context in which your data will be used and the encoding that will be expected. This is especially important when transmitting data between different components, or when generating outputs that can contain multiple encodings at the same time, such as web pages or multi-part mail messages. Study all expected communication protocols and data representations to determine the required encoding strategies. For any data that will be output to another web page, especially any data that was received from external inputs, use the appropriate encoding on all non-alphanumeric characters. Consult the XSS Prevention Cheat Sheet for more details on the types of encoding and escaping that are needed. Phase: Architecture and Design For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server. If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated. Phase: Implementation For every web page that is generated, use and specify a character encoding such as ISO-8859-1 or UTF-8. When an encoding is not specified, the web browser may choose a different encoding by guessing which encoding is actually being used by the web page. This can cause the web browser to treat certain sequences as special, opening up the client to subtle XSS attacks. See CWE-116 for more mitigations related to encoding/escaping. To help mitigate XSS attacks against the user's session cookie, set the session cookie to be HttpOnly. In browsers that support the HttpOnly feature (such as more recent versions of Internet Explorer and Firefox), this attribute can prevent the user's session cookie from being accessible to malicious client-side scripts that use document.cookie. This is not a complete solution, since HttpOnly is not supported by all browsers. More importantly, XMLHttpRequest and other powerful browser technologies provide read access to HTTP headers, including the Set-Cookie header in which the HttpOnly flag is set. Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency

across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

Ensure that you perform input validation at well-defined interfaces within the application. This will help protect the application even if a component is reused or moved elsewhere.

Reference	https://owasp.org/www-community/attacks/xss/ https://cwe.mitre.org/data/definitions/79.html
CWE Id	79
WASC Id	8
Plugin Id	40012
High	Path Traversal The Path Traversal attack technique allows an attacker access to files, directories, and commands that potentially reside outside the web document root directory. An attacker may manipulate a URL in such a way that the web site will execute or reveal the contents of arbitrary files anywhere on the web server. Any device that exposes an HTTP-based interface is potentially vulnerable to Path Traversal. Most web sites restrict user access to a specific portion of the file-system, typically called the "web document root" or "CGI root" directory. These directories contain the files intended for user access and the executable necessary to drive web application functionality. To access files or execute commands anywhere on the file-system, Path Traversal attacks will utilize the ability of special-characters sequences. The most basic Path Traversal attack uses the "../" special-character sequence to alter the resource location requested in the URL. Although most popular web servers will prevent this technique from escaping the web document root, alternate encodings of the "../" sequence may help bypass the security filters. These method variations include valid and invalid Unicode-encoding ("..%u2216" or "..%c0%af") of the forward slash character, backslash characters ("..\") on Windows-based servers, URL encoded characters "%2e%2e%2f"), and double URL encoding ("..%255c") of the backslash character. Even if the web server properly restricts Path Traversal attempts in the URL path, a web application itself may still be vulnerable due to improper handling of user-supplied input. This is a common problem of web applications that use template mechanisms or load static text from files. In variations of the attack, the original URL parameter value is substituted with the file name of one of the web application's dynamic scripts. Consequently, the results can reveal source code because the file is interpreted as text instead of an executable script. These techniques often employ additional special characters such as the dot (".") to reveal the listing of the current working directory, or "%00" NULL characters in order to bypass rudimentary file extension checks.
Description	
URL	http://secureapp:5000/todos/add
Node Name	http://secureapp:5000/todos/add ()(task)
Method	POST
Parameter	task
Attack	add
Evidence	
Other Info	
Instances	1
Solution	Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright.

When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

For filenames, use stringent allow lists that limit the character set to be used. If feasible, only allow a single "." character in the filename to avoid weaknesses, and exclude directory separators such as "/". Use an allow list of allowable file extensions.

Warning: if you attempt to cleanse your data, then do so that the end result is not in the form that can be dangerous. A sanitizing mechanism can remove characters such as '.' and ';' which may be required for some exploits. An attacker can try to fool the sanitizing mechanism into "cleaning" data into a dangerous form. Suppose the attacker injects a '.' inside a filename (e.g. "sensi.tiveFile") and the sanitizing mechanism removes the character resulting in the valid filename, "sensitiveFile". If the input data are now assumed to be safe, then the file may be compromised.

Inputs should be decoded and canonicalized to the application's current internal representation before being validated. Make sure that your application does not decode the same input twice. Such errors could be used to bypass allow list schemes by introducing dangerous inputs after they have been checked.

Use a built-in path canonicalization function (such as realpath() in C) that produces the canonical version of the pathname, which effectively removes ".." sequences and symbolic links.

Run your code using the lowest privileges that are required to accomplish the necessary tasks. If possible, create isolated accounts with limited privileges that are only used for a single task. That way, a successful attack will not immediately give the attacker access to the rest of the software or its environment. For example, database applications rarely need to run as the database administrator, especially in day-to-day operations.

When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input values (such as numeric IDs) to the actual filenames or URLs, and reject all other inputs.

Run your code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by your software.

OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general, managed code may provide some protection. For example, java.io.FilePermission in the Java SecurityManager allows you to specify restrictions on file operations.

This may not be a feasible solution, and it only limits the impact to the operating system; the rest of your application may still be subject to compromise.

Reference	https://owasp.org/www-community/attacks/Path_Traversal https://cwe.mitre.org/data/definitions/22.html
CWE Id	22
WASC Id	33
Plugin Id	6

High SQL Injection

Description SQL injection may be possible.

URL	http://secureapp:5000/post/new
Node Name	http://secureapp:5000/post/new()(content,title)
Method	POST
Parameter	title
Attack	ZAP'

Evidence	HTTP/1.1 500 INTERNAL SERVER ERROR
Other Info	
URL	http://secureapp:5000/register
Node Name	http://secureapp:5000/register()(email,password,username)
Method	POST
Parameter	username
Attack	ZAP'
Evidence	HTTP/1.1 500 INTERNAL SERVER ERROR
Other Info	
URL	http://secureapp:5000/todos/add
Node Name	http://secureapp:5000/todos/add()(task)
Method	POST
Parameter	task
Attack	ZAP'
Evidence	HTTP/1.1 500 INTERNAL SERVER ERROR
Other Info	
URL	http://secureapp:5000/login
Node Name	http://secureapp:5000/login()(password,username)
Method	POST
Parameter	username
Attack	ZAP' AND '1'='1' --
Evidence	
Other Info	The page results were successfully manipulated using the boolean conditions [ZAP' AND '1'='1' --] and [ZAP' AND '1'='2' --] The parameter value being modified was stripped from the HTML output for the purposes of the comparison. Data was returned for the original parameter. The vulnerability was detected by successfully restricting the data originally returned, by manipulating the parameter.
Instances	4
Solution	Do not trust client side input, even if there is client side validation in place. In general, type check all data on the server side. If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?' If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries. If database Stored Procedures can be used, use them. Do <i>*not*</i> concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality!

Do not create dynamic SQL queries using simple string concatenation.

Escape all data received from the client.

Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input.

Apply the principle of least privilege by using the least privileged database user possible.

In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact.

Grant the minimum database access that is necessary for the application.

Reference	https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
CWE Id	89
WASC Id	19
Plugin Id	40018

High SQL Injection - Authentication Bypass

Description SQL injection may be possible on a login page, potentially allowing the application's authentication mechanism to be bypassed

URL	http://secureapp:5000/login
Node Name	http://secureapp:5000/login ()(password,username)
Method	POST
Parameter	username
Attack	ZAP' AND '1'='1' --
Evidence	
Other Info	

Instances	1
	Do not trust client side input, even if there is client side validation in place.
	In general, type check all data on the server side.
	If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?'
	If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries.
	If database Stored Procedures can be used, use them.

Solution	Do *not* concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality!
	Do not create dynamic SQL queries using simple string concatenation.
	Escape all data received from the client.
	Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input.
	Apply the principle of least privilege by using the least privileged database user possible.
	In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact.
	Grant the minimum database access that is necessary for the application.

Reference	https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
-----------	---

CWE Id [89](#)
WASC Id 19
Plugin Id [40018](#)

Medium **Absence of Anti-CSRF Tokens**

No Anti-CSRF tokens were found in a HTML submission form.

A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf.

Description CSRF attacks are effective in a number of situations, including:

- * The victim has an active session on the target site.
- * The victim is authenticated via HTTP auth on the target site.
- * The victim is on the same local network as the target site.

CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy.

URL <http://secureapp:5000/login>

Node Name http://secureapp:5000/login

Method GET

Parameter

Attack

Evidence <form method="POST">

Other Info No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "password" "username"].

URL <http://secureapp:5000/post/new>

Node Name http://secureapp:5000/post/new

Method GET

Parameter

Attack

Evidence <form method="POST">

Other Info No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "title"].

URL <http://secureapp:5000/register>

Node Name http://secureapp:5000/register

Method	GET
Parameter	
Attack	
Evidence	<form method="POST">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "email" "password" "username"].
URL	http://secureapp:5000/todos
Node Name	http://secureapp:5000/todos
Method	GET
Parameter	
Attack	
Evidence	<form method="POST" action="/todos/add">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "task"].
URL	http://secureapp:5000/login
Node Name	http://secureapp:5000/login ()(password,username)
Method	POST
Parameter	
Attack	
Evidence	<form method="POST">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "password" "username"].
Instances	5
Solution	Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, use anti-CSRF packages such as the OWASP CSRFGuard. Phase: Implementation Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script. Phase: Architecture and Design Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330). Note that this can be bypassed using XSS. Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation.

Note that this can be bypassed using XSS.

Use the ESAPI Session Management control.

This control includes a component for CSRF.

Do not use the GET method for any request that triggers a state change.

Phase: Implementation

Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.

Reference	https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/352.html
CWE Id	352
WASC Id	9
Plugin Id	10202

Medium

Content Security Policy (CSP) Header Not Set

Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://secureapp:5000
Node Name	http://secureapp:5000
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://secureapp:5000/admin
Node Name	http://secureapp:5000/admin
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://secureapp:5000/post/new
Node Name	http://secureapp:5000/post/new
Method	GET
Parameter	
Attack	

Evidence	
Other Info	
URL	http://secureapp:5000/register
Node Name	http://secureapp:5000/register
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://secureapp:5000/sitemap.xml
Node Name	http://secureapp:5000/sitemap.xml
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/
Reference	https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp https://caniuse.com/#feat=contentsecuritypolicy https://content-security-policy.com/
CWE Id	693
WASC Id	15
Plugin Id	10038
Medium	Missing Anti-clickjacking Header
Description	The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
URL	http://secureapp:5000
Node Name	http://secureapp:5000
Method	GET
Parameter	x-frame-options
Attack	
Evidence	
Other Info	
URL	http://secureapp:5000/admin

Node Name	http://secureapp:5000/admin
Method	GET
Parameter	x-frame-options
Attack	
Evidence	
Other Info	
URL	http://secureapp:5000/post/new
Node Name	http://secureapp:5000/post/new
Method	GET
Parameter	x-frame-options
Attack	
Evidence	
Other Info	
URL	http://secureapp:5000/register
Node Name	http://secureapp:5000/register
Method	GET
Parameter	x-frame-options
Attack	
Evidence	
Other Info	
URL	http://secureapp:5000/login
Node Name	http://secureapp:5000/login ()(password,username)
Method	POST
Parameter	x-frame-options
Attack	
Evidence	
Other Info	
Instances	Systemic Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app.
Solution	If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
CWE Id	1021
WASC Id	15
Plugin Id	10020

Low	Cookie without SameSite Attribute
Description	A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.
URL	http://secureapp:5000/login
Node Name	http://secureapp:5000/login ()(password,username)
Method	POST
Parameter	session
Attack	
Evidence	Set-Cookie: session
Other Info	
Instances	1
Solution	Ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.
Reference	https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-cookie-same-site
CWE Id	1275
WASC Id	13
Plugin Id	10054
Low	Server Leaks Version Information via "Server" HTTP Response Header Field
Description	The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.
URL	http://secureapp:5000/admin
Node Name	http://secureapp:5000/admin
Method	GET
Parameter	
Attack	
Evidence	Werkzeug/3.0.4 Python/3.12.13
Other Info	
URL	http://secureapp:5000/robots.txt
Node Name	http://secureapp:5000/robots.txt
Method	GET
Parameter	
Attack	
Evidence	Werkzeug/3.0.4 Python/3.12.13
Other Info	
URL	http://secureapp:5000/sitemap.xml
Node Name	http://secureapp:5000/sitemap.xml
Method	GET

Parameter	
Attack	
Evidence	Werkzeug/3.0.4 Python/3.12.13
Other Info	
URL	http://secureapp:5000/post/new
Node Name	http://secureapp:5000/post/new()(content,title)
Method	POST
Parameter	
Attack	
Evidence	Werkzeug/3.0.4 Python/3.12.13
Other Info	
URL	http://secureapp:5000/register
Node Name	http://secureapp:5000/register()(email,password,username)
Method	POST
Parameter	
Attack	
Evidence	Werkzeug/3.0.4 Python/3.12.13
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.
Reference	https://httpd.apache.org/docs/current/mod/core.html#servertokens https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10) https://www.troyhunt.com/shhh-dont-let-your-response-headers/
CWE Id	497
WASC Id	13
Plugin Id	10036
Low	X-Content-Type-Options Header Missing
Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	http://secureapp:5000
Node Name	http://secureapp:5000
Method	GET
Parameter	x-content-type-options
Attack	
Evidence	

Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type.	
	At "High" threshold this scan rule will not alert on client or server error responses.	
URL	http://secureapp:5000/admin	
Node Name	http://secureapp:5000/admin	
Method	GET	
Parameter	x-content-type-options	
Attack		
Evidence		
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type.	
	At "High" threshold this scan rule will not alert on client or server error responses.	
URL	http://secureapp:5000/register	
Node Name	http://secureapp:5000/register	
Method	GET	
Parameter	x-content-type-options	
Attack		
Evidence		
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type.	
	At "High" threshold this scan rule will not alert on client or server error responses.	
URL	http://secureapp:5000/todos	
Node Name	http://secureapp:5000/todos	
Method	GET	
Parameter	x-content-type-options	
Attack		
Evidence		
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type.	
	At "High" threshold this scan rule will not alert on client or server error responses.	
URL	http://secureapp:5000/login	
Node Name	http://secureapp:5000/login ()(password,username)	
Method	POST	
Parameter	x-content-type-options	
Attack		

Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
Instances	Systemic Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages.
Solution	If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security-Headers
CWE Id	693
WASC Id	15
Plugin Id	10021
Informational	Authentication Request Identified
Description	The given request has been identified as an authentication request. The 'Other Info' field contains a set of key=value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to "Auto-Detect" then this rule will change the authentication to match the request identified.
URL	http://secureapp:5000/login
Node Name	http://secureapp:5000/login ()(password,username)
Method	POST
Parameter	username
Attack	
Evidence	password userParam=username
Other Info	userValue=admin passwordParam=password
Instances	1
Solution	This is an informational alert rather than a vulnerability and so there is nothing to fix.
Reference	https://www.zaproxy.org/docs/desktop/addons/authentication-helper/auth-req-id/
CWE Id	
WASC Id	
Plugin Id	10111
Informational	Session Management Response Identified
Description	The given response has been identified as containing a session management token. The 'Other Info' field contains a set of header tokens that can be used in the Header Based Session Management Method. If the request is in a context which has a Session Management Method set to "Auto-Detect" then this rule will change the session management to use the tokens identified.
URL	http://secureapp:5000/login

Node Name	http://secureapp:5000/login ()(password,username)
Method	POST
Parameter	session
Attack	
Evidence	session
Other Info	cookie:session
URL	http://secureapp:5000/login
Node Name	http://secureapp:5000/login ()(password,username)
Method	POST
Parameter	session
Attack	
Evidence	session
Other Info	cookie:session
Instances	2
Solution	This is an informational alert rather than a vulnerability and so there is nothing to fix.
Reference	https://www.zaproxy.org/docs/desktop/addons/authentication-helper/session-mgmt-id/
CWE Id	
WASC Id	
Plugin Id	10112

Sequence Details

With the associated active scan results.